



UNIVERSITÉ D'AIX-MARSEILLE
FORMATION MASTER 2 : MASSPOP

Données Massives

Titre du Projet :
Analyse des retards des vols à New York (2013)

Étudiants :
Arezki HASSAK
Mathieu GRONDIN

Enseignante :
Laurence Reboul

Année universitaire : 2024–2025

Analyse des retards des vols à New York (2013)

Table of Contents

1	Introduction	2
2	Chargement des packages	2
3	Partie A : Manipulation de données avec dplyr, data.table et ggplot2	3
3.1	Chargement des données et jointure flights-airlines	3
3.2	Ajout des colonnes distance, vitesse, indicateurs de retard.....	5
3.3	Destinations desservies par au moins deux compagnies	6
3.4	Résumé statistique	7
3.5	les vols arrivés avec plus de deux heures de retard, mais qui ne sont pas partis en retard.....	8
3.6	Un fichier donnant, pour chaque compagnie : le nombre de vols effectués, le pourcentage de ces vols retardés à l'arrivée, le retard moyen et le retard maximal à l'arrivée. Quelles sont les 10 compagnies ayant les plus grands pourcentages de retards à l'arrivée ?.....	10
3.7	Un tableau donnant pour chaque mois, la proportion relative de vols retardés et de vols non retardés à l'arrivée. Ce tableau sera présenté par mois (en ligne) × indicatrice de retard (en colonne). Représenter ces proportions avec un graphique adapté.....	12
3.8	Un tableau contenant les retards moyens (en cas de retard) par compagnie (en ligne) × mois (en colonne).....	14
3.9	Pour chaque destination et chaque vol, calculez le retard en vol (time.delay) = temps de vol par rapport au temps de vol le plus court vers cette destination et sauvegardez cette nouvelle variable dans la table, puis pour chaque destination, donnez le vol qui a eu le plus de retard en vol avec son origine et sa destination.....	17
3.10	Donner par mois la destination la plus retardée à l'arrivée. On affichera son retard moyen à l'arrivée et au départ, le nombre et le pourcentage de vols retardés à l'arrivée, la vitesse moyenne de vol en km, la distance moyenne parcourue en km ainsi que le retard en vol moyen (cf 9)).....	18
3.11	Un graphique permettant de montrer comment le retard moyen à l'arrivée varie en fonction du mois et de la destination.....	20
3.12	Pour un certain nombre de vols, la valeur du retard à l'arrivée est manquante. Ces valeurs manquantes sont probablement des vols annulés. Calculer et représenter le nombre de vols annulés par jour. Y a-t-il une tendance ?.....	23
3.13	On utilise la table planes pour avoir des informations sur les avions utilisés. Joindre la table principale à la table planes pour inclure l'année de fabrication (ancienneté) de l'avion. À l'aide de tableaux et/ou graphiques, répondre aux questions suivantes : existe-t-il une relation	25

3.13.1	Relation entre le retard au départ et à l'arrivée.....	26
3.13.2	Relation entre l'année de fabrication et le retard à l'arrivée	27
3.13.3	Relation entre le retard en vol (time_delay) et le retard à l'arrivée.....	28
3.13.4	Relation entre la destination et le retard à l'arrivée.....	29
3.14	On souhaite expliquer le retard à l'arrivée par les conditions météorologiques au départ.Effectuer une jointure de la table principale à la table weather et calculer la température en degrés Celsius (on rappelle que $t^F = 95t^C + 32$.)	31
3.15	Le lien entre les variables météo au départ et le retard à l'arrivée. On pourra effectuer par exemple une régression linéaire expliquant le retard à l'arrivée, ou une régression logistique expliquant l'indicatrice de retard à l'arrivée (On pourra se limiter à une seule syntaxe entre dplyr et data.table).....	34
3.16	Les retards sont généralement corrélés dans le temps : même une fois que le problème à l'origine du retard initial a été résolu, les vols ultérieurs sont retardés pour permettre aux vols antérieurs de partir. En utilisant la fonction lag(), étudiez comment le retard d'un vol est lié au retard du vol précédent (à ne faire qu'avec dplyr).	35
4	Partie B : Programmation MapReduce	37
4.1	Écrire un programme MapReduce qui calcule le nombre de vols annulés par compagnie	37
4.2	Retard maximal par compagnie	38
4.3	Compagnie avec le plus grand % de vols retardés mensuels	39

1 Introduction

Ce projet vise à analyser les retards des vols ayant quitté New York (JFK, LGA, EWR) en 2013, à partir des jeux de données fournis par le package `nycflights13`. Nous utiliserons les packages **dplyr**, **data.table**, **ggplot2** ainsi que des techniques de programmation **MapReduce**.

2 Chargement des packages

```
#install.packages("nycflights13")
#install.packages(c("ggplot2", "dplyr", "tidyr", "data.table"))
library(nycflights13)
library(dplyr)

##
## Attachement du package : 'dplyr'
```

```
## Les objets suivants sont masqués depuis 'package:stats':
##
##     filter, lag

## Les objets suivants sont masqués depuis 'package:base':
##
##     intersect, setdiff, setequal, union

library(data.table)

##
## Attachement du package : 'data.table'

## Les objets suivants sont masqués depuis 'package:dplyr':
##
##     between, first, last

library(ggplot2)
```

3 Partie A : Manipulation de données avec dplyr, data.table et ggplot2

3.1 Chargement des données et jointure flights-airlines

Avec data.table

```
# Conversion des tables en data.table
fl_dt <- as.data.table(flights)
al_dt <- as.data.table(airlines)

# Jointure : On ajoute les noms des compagnies
setkey(fl_dt, carrier) # Création de la clé pour accélérer la jointure
jn_dt <- al_dt[fl_dt, on="carrier"]

# Affichage d'un aperçu
head(jn_dt)
```

	carrier	name	year	month	day	dep_time	sched_dep_time
	<char>	<char>	<int>	<int>	<int>	<int>	<int>
## 1:	9E	Endeavor Air Inc.	2013	1	1	810	810
## 2:	9E	Endeavor Air Inc.	2013	1	1	1451	1500
## 3:	9E	Endeavor Air Inc.	2013	1	1	1452	1455
## 4:	9E	Endeavor Air Inc.	2013	1	1	1454	1500
## 5:	9E	Endeavor Air Inc.	2013	1	1	1507	1515
## 6:	9E	Endeavor Air Inc.	2013	1	1	1530	1530

```
##     dep_delay arr_time sched_arr_time arr_delay flight tailnum origin
##     <num>    <int>          <int>      <num>  <int>  <char> <char>
##     <char>
## 1:         0     1048           1037        11   3538  N915XJ   JFK
```

```

MSP
## 2:      -9      1634      1636      -2    4105    N8444F    JFK
IAD
## 3:      -3      1637      1639      -2    3295    N920XJ    JFK
BUF
## 4:      -6      1635      1636      -1    3843    N8409N    JFK
SYR
## 5:      -8      1651      1656      -5    3792    N8631E    JFK
ROC
## 6:       0      1650      1655      -5    3369    N913XJ    JFK
BWI
##      air_time distance  hour minute      time_hour
##      <num>    <num> <num>  <num>      <POSc>
## 1:      189     1029     8    10 2013-01-01 08:00:00
## 2:       57      228    15     0 2013-01-01 15:00:00
## 3:       68      301    14    55 2013-01-01 14:00:00
## 4:       57      209    15     0 2013-01-01 15:00:00
## 5:       66      264    15    15 2013-01-01 15:00:00
## 6:       40      184    15    30 2013-01-01 15:00:00

```

Avec dplyr

```

# Jointure simple avec dplyr
jn <- left_join(flights, airlines, by = "carrier")

# Affichage d'un aperçu
head(jn)

## # A tibble: 6 × 20
##   year month   day dep_time sched_dep_time dep_delay arr_time
##   <int> <int> <int>   <int>         <int>         <dbl>   <int>
## 1  2013     1     1     517             515           2     830
## 2  2013     1     1     533             529           4     850
## 3  2013     1     1     542             540           2     923
## 4  2013     1     1     544             545          -1    1004
## 5  2013     1     1     554             600          -6     812
## 6  2013     1     1     554             558          -4     740
## # [i] 12 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance
## #   <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dtm>, name <chr>

```

3.2 Ajout des colonnes distance, vitesse, indicateurs de retard

Avec data.table

```
# On suppose que jn_dt est déjà un data.table
# Ajout des colonnes en dur
jn_dt[, dist_km := distance * 1.60934] # KM
jn_dt[, vit_km := dist_km / (air_time / 60)] # Vitesse en km/h
jn_dt[, arr_del := ifelse(!is.na(arr_delay), arr_delay >= 10, NA)] # Retard à l'arrivée
jn_dt[, dep_del := ifelse(!is.na(dep_delay), dep_delay >= 10, NA)] # Retard au départ

# Aperçu
head(jn_dt[, .(distance, dist_km, air_time, vit_km, arr_delay, arr_del, dep_delay, dep_del)])
```

	distance	dist_km	air_time	vit_km	arr_delay	arr_del	dep_delay
1:	1029	1656.0109	189	525.7177	11	TRUE	0
2:	228	366.9295	57	386.2416	-2	FALSE	-9
3:	301	484.4113	68	427.4218	-2	FALSE	-3
4:	209	336.3521	57	354.0548	-1	FALSE	-6
5:	264	424.8658	66	386.2416	-5	FALSE	-8
6:	184	296.1186	40	444.1778	-5	FALSE	0

Avec dplyr

```
jn <- jn %>%
  mutate(
    dist_km = distance * 1.60934,
    vit_km = dist_km / (air_time / 60), # Vitesse
    arr_del = if_else(!is.na(arr_delay), arr_delay >= 10, NA), # Retard à l'arrivée
    dep_del = if_else(!is.na(dep_delay), dep_delay >= 10, NA) # Retard au départ
  )

# Aperçu
jn %>% select(distance, dist_km, air_time, vit_km, arr_delay, arr_del, dep_delay, dep_del) %>% head()
```

```
## # A tibble: 6 × 8
##   distance dist_km air_time vit_km arr_delay arr_del dep_delay dep_del
##   <dbl>   <dbl>   <dbl>  <dbl>   <dbl> <lgl>     <dbl> <lgl>
## 1    1400    2253.     227    596.     11 TRUE         2 FALSE
## 2    1416    2279.     227    602.     20 TRUE         4 FALSE
## 3    1089    1753.     160    657.     33 TRUE         2 FALSE
## 4    1576    2536.     183    832.    -18 FALSE        -1 FALSE
## 5     762    1226.     116    634.    -25 FALSE        -6 FALSE
## 6     719    1157.     150    463.     12 TRUE         -4 FALSE
```

3.3 Destinations desservies par au moins deux compagnies

Avec data.table

```
# On cherche pour chaque destination Le nombre de compagnies différentes
res_dt <- jn_dt[, .(nb_comp = uniqueN(name)), by = dest][nb_comp >= 2]
```

```
# Affichage des destinations concernées
```

```
head(res_dt)
```

```
##      dest nb_comp
##   <char>   <int>
## 1:   MSP      6
## 2:   IAD      6
## 3:   BUF      4
## 4:   SYR      3
## 5:   ROC      3
## 6:   BWI      4
```

Avec dplyr

```
# Même opération avec dplyr
```

```
res_dplyr <- jn %>%
  group_by(dest) %>%
  summarise(nb_comp = n_distinct(name)) %>%
  filter(nb_comp >= 2)
```

```
# Affichage
```

```
head(res_dplyr)
```

```
## # A tibble: 6 × 2
##   dest nb_comp
##   <chr>   <int>
## 1 ATL      7
## 2 AUS      6
## 3 AVL      2
## 4 BDL      2
## 5 BGR      2
## 6 BNA      5
```

3.4 Résumé statistique

Avec data.table

```
# Résumé statistique avec data.table
stat_summary_dt <- jn_dt[, .(
  # Proportion de données manquantes
  missing_arr_delay = mean(is.na(arr_delay)),
  missing_dep_delay = mean(is.na(dep_delay)),

  # Retard moyen et écart-type à l'arrivée, en cas de retard
  mean_arr_delay = mean(arr_delay[arr_delay > 0], na.rm = TRUE),
  sd_arr_delay = sd(arr_delay[arr_delay > 0], na.rm = TRUE),

  # Proportion de retards à l'arrivée
  prop_arr_delay = mean(arr_del == 1, na.rm = TRUE)
)]

# Afficher le résumé
print(stat_summary_dt)

##      missing_arr_delay missing_dep_delay mean_arr_delay sd_arr_delay
##                <num>                <num>                <num>                <num>
## 1:          0.02800081          0.02451184          40.3425          53.34633
##      prop_arr_delay
##                <num>
## 1:          0.2901945
```

Avec dplyr

```
# Résumé statistique avec dplyr
stat_summary <- jn %>%
  summarise(
    # Proportion de données manquantes
    missing_arr_delay = mean(is.na(arr_delay)),
    missing_dep_delay = mean(is.na(dep_delay)),

    # Retard moyen et écart-type à l'arrivée, en cas de retard
    mean_arr_delay = mean(arr_delay[arr_delay > 0], na.rm = TRUE),
    sd_arr_delay = sd(arr_delay[arr_delay > 0], na.rm = TRUE),

    # Proportion de retards à l'arrivée (retard >= 10 min)
    prop_arr_delay = mean(arr_del == 1, na.rm = TRUE)
  )

# Afficher le résumé
print(stat_summary)

## # A tibble: 1 × 5
##   missing_arr_delay missing_dep_delay mean_arr_delay sd_arr_delay
```



```
prop_arr_delay
##           <dbl>           <dbl>           <dbl>           <dbl>
<dbl>
## 1           0.0280           0.0245           40.3           53.3
0.290
```

3.5 les vols arrivés avec plus de deux heures de retard, mais qui ne sont pas partis en retard

Avec data.table

```
# Vols arrivés avec >2h de retard mais partis sans retard (ou en avance)
vols_retard_dt <- jn_dt[arr_delay > 120 & dep_delay <= 0]
```

```
# Affichage simple
```

```
head(vols_retard_dt)
```

```
##      carrier          name  year month  day dep_time
sched_dep_time
##      <char>          <char> <int> <int> <int>    <int>
<int>
## 1:      AA American Airlines Inc.  2013   10   7    1357
1359
## 2:      AA American Airlines Inc.  2013    4  18    558
600
## 3:      AA American Airlines Inc.  2013    4  18    655
700
## 4:      AA American Airlines Inc.  2013    6  14   1708
1710
## 5:      B6      JetBlue Airways  2013   10  16    657
700
## 6:      B6      JetBlue Airways  2013    6  30   1423
1425
##      dep_delay arr_time sched_arr_time arr_delay flight tailnum origin
dest
##      <num>    <int>          <int>      <num> <int> <char> <char>
<char>
## 1:         -2    1858            1654        124  1151  N3CMAA  LGA
DFW
## 2:         -2    1149             850        179   707  N3EXAA  LGA
DFW
## 3:         -5    1213             950        143  2083  N565AA  EWR
DFW
## 4:         -2    2227            2015        132   181  N320AA  JFK
LAX
## 5:         -3    1258            1056        122    3  N703JB  JFK
SJU
## 6:         -2    1816            1554        142  2402  N206JB  JFK
BUF
##      air_time distance  hour minute      time_hour  dist_km  vit_km
```

```
##      <num>      <num> <num> <num>      <POSc>      <num>      <num>
## 1:      192      1389    13     59 2013-10-07 13:00:00 2235.3733 698.5541
## 2:      234      1389     6      0 2013-04-18 06:00:00 2235.3733 573.1726
## 3:      230      1372     7      0 2013-04-18 07:00:00 2208.0145 576.0038
## 4:      334      2475    17     10 2013-06-14 17:00:00 3983.1165 715.5299
## 5:      225      1598     7      0 2013-10-16 07:00:00 2571.7253 685.7934
## 6:       80       301    14     25 2013-06-30 14:00:00 484.4113 363.3085
##   arr_del dep_del
##   <lgcl> <lgcl>
## 1:   TRUE  FALSE
## 2:   TRUE  FALSE
## 3:   TRUE  FALSE
## 4:   TRUE  FALSE
## 5:   TRUE  FALSE
## 6:   TRUE  FALSE
```

Avec dplyr

```
library(dplyr)

# Vols arrivés avec >2h de retard mais partis à l'heure
vols_retard_dplyr <- jn %>%
  filter(arr_delay > 120, dep_delay <= 0)

# Affichage simple
head(vols_retard_dplyr)

## # A tibble: 6 × 24
##   year month   day dep_time sched_dep_time dep_delay arr_time
##   <int> <int> <int>   <int>         <int>      <dbl>   <int>
## 1  2013     1    27    1419           1420        -1    1754
## 2  2013    10     7    1350           1350         0    1736
## 3  2013    10     7    1357           1359        -2    1858
## 4  2013    10    16     657            700        -3    1258
## 5  2013    11     1     658            700        -2    1329
## 6  2013     3    18    1844           1847        -3     39
## # [i] 16 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance
## #   <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dtm>, name <chr>, dist_km <dbl>,
## #   vit_km <dbl>, arr_del <lgcl>, dep_del <lgcl>
```

3.6 Un fichier donnant, pour chaque compagnie : le nombre de vols effectués, le pourcentage de ces vols retardés à l'arrivée, le retard moyen et le retard maximal à l'arrivée. Quelles sont les 10 compagnies ayant les plus grands pourcentages de retards à l'arrivée ?

Avec data.table

```
# Statistiques par compagnie
stats_carrier_dt <- jn_dt[, .(
  n_vols = .N,
  pourc_retard = mean(arr_del, na.rm = TRUE) * 100,
  retard_moyen = mean(arr_delay[arr_delay > 0], na.rm = TRUE),
  retard_max = max(arr_delay, na.rm = TRUE)
), by = carrier]
```

```
# Voir les 10 compagnies avec le plus grand % de retards
top10_retards_dt <- stats_carrier_dt[order(-pourc_retard)][1:10]
```

```
# Afficher
stats_carrier_dt
```

	carrier	n_vols	pourc_retard	retard_moyen	retard_max
	<char>	<int>	<num>	<num>	<num>
## 1:	9E	18460	29.11414	49.27271	744
## 2:	AA	32729	23.35744	38.26555	1007
## 3:	AS	714	19.04090	34.36508	198
## 4:	B6	54635	31.63981	40.00906	497
## 5:	DL	48110	23.21751	37.74356	931
## 6:	EV	54173	36.57940	48.26858	577
## 7:	F9	685	44.93392	47.57908	834
## 8:	FL	3260	42.58268	41.09446	572
## 9:	HA	342	19.00585	35.03093	1272
## 10:	MQ	26397	33.53437	37.85205	1127
## 11:	OO	32	24.13793	60.60000	157
## 12:	UA	58665	27.06206	36.65098	455
## 13:	US	20536	23.49352	29.01157	492
## 14:	VX	5162	22.94762	43.84708	676
## 15:	WN	12275	30.73730	40.74755	453
## 16:	YV	601	37.31618	51.08140	381

```
top10_retards_dt
```

	carrier	n_vols	pourc_retard	retard_moyen	retard_max
	<char>	<int>	<num>	<num>	<num>
## 1:	F9	685	44.93392	47.57908	834
## 2:	FL	3260	42.58268	41.09446	572
## 3:	YV	601	37.31618	51.08140	381
## 4:	EV	54173	36.57940	48.26858	577
## 5:	MQ	26397	33.53437	37.85205	1127
## 6:	B6	54635	31.63981	40.00906	497

## 7:	WN	12275	30.73730	40.74755	453
## 8:	9E	18460	29.11414	49.27271	744
## 9:	UA	58665	27.06206	36.65098	455
## 10:	00	32	24.13793	60.60000	157

Avec dplyr

```
# Statistiques par compagnie
stats_carrier_dplyr <- jn %>%
  group_by(carrier) %>%
  summarise(
    n_vols = n(),
    pourc_retard = mean(arr_del, na.rm = TRUE) * 100,
    retard_moyen = mean(arr_delay[arr_delay > 0], na.rm = TRUE),
    retard_max = max(arr_delay, na.rm = TRUE),
    .groups = 'drop'
  )
```

```
# Voir les 10 compagnies avec le plus grand % de retards
top10_retards_dplyr <- stats_carrier_dplyr %>%
  arrange(desc(pourc_retard)) %>%
  slice(1:10)
```

Afficher

```
stats_carrier_dplyr
```

```
## # A tibble: 16 × 5
##   carrier n_vols pourc_retard retard_moyen retard_max
##   <chr>   <int>      <dbl>      <dbl>      <dbl>
## 1 9E      18460      29.1        49.3        744
## 2 AA      32729      23.4        38.3       1007
## 3 AS        714      19.0        34.4        198
## 4 B6      54635      31.6        40.0        497
## 5 DL      48110      23.2        37.7        931
## 6 EV      54173      36.6        48.3        577
## 7 F9        685      44.9        47.6        834
## 8 FL       3260      42.6        41.1        572
## 9 HA        342      19.0        35.0       1272
## 10 MQ     26397      33.5        37.9       1127
## 11 00        32      24.1        60.6        157
## 12 UA      58665      27.1        36.7        455
## 13 US     20536      23.5        29.0        492
## 14 VX       5162      22.9        43.8        676
## 15 WN      12275      30.7        40.7        453
## 16 YV        601      37.3        51.1        381
```

```
top10_retards_dplyr
```

```
## # A tibble: 10 × 5
##   carrier n_vols pourc_retard retard_moyen retard_max
##   <chr>   <int>      <dbl>      <dbl>      <dbl>
```

##	1	F9	685	44.9	47.6	834
##	2	FL	3260	42.6	41.1	572
##	3	YV	601	37.3	51.1	381
##	4	EV	54173	36.6	48.3	577
##	5	MQ	26397	33.5	37.9	1127
##	6	B6	54635	31.6	40.0	497
##	7	WN	12275	30.7	40.7	453
##	8	9E	18460	29.1	49.3	744
##	9	UA	58665	27.1	36.7	455
##	10	00	32	24.1	60.6	157

3.7 Un tableau donnant pour chaque mois, la proportion relative de vols retardés et de vols non retardés à l'arrivée. Ce tableau sera présenté par mois (en ligne) × indicatrice de retard (en colonne). Représenter ces proportions avec un graphique adapté.

Avec data.table

```
# Filtrer les lignes valides
jn_dt_valid <- jn_dt[!is.na(arr_del)]

# Comptage par mois et indicatrice
tab_mois_dt <- jn_dt_valid[, .N, by = .(month, arr_del)]
tab_mois_dt[, prop := N / sum(N), by = month]

# Tableau croisé avec dcast
tab_croise_dt <- dcast(tab_mois_dt, month ~ arr_del, value.var = "prop")
setnames(tab_croise_dt, c("month", "Non_retardé", "Retardé")) # Renommer les
colonnes logiques
print(tab_croise_dt)
```

```
## Key: <month>
##      month Non_retardé   Retardé
##      <int>      <num>      <num>
##  1:      1  0.7132359 0.2867641
##  2:      2  0.7077210 0.2922790
##  3:      3  0.7201634 0.2798366
##  4:      4  0.6649978 0.3350022
##  5:      5  0.7351394 0.2648606
##  6:      6  0.6435826 0.3564174
##  7:      7  0.6353515 0.3646485
##  8:      8  0.7081305 0.2918695
##  9:      9  0.8330618 0.1669382
## 10:     10  0.7790901 0.2209099
## 11:     11  0.7719773 0.2280227
## 12:     12  0.6043671 0.3956329
```

Avec dplyr

```
library(tidyr)

# Filtrer les lignes valides
tab_mois_dplyr <- jn %>%
  filter(!is.na(arr_del)) %>%
  group_by(month, arr_del) %>%
  summarise(N = n(), .groups = "drop") %>%
  group_by(month) %>%
  mutate(prop = N / sum(N)) %>%
  select(month, arr_del, prop) %>%
  ungroup() %>%
  mutate(arr_del = ifelse(arr_del, "Retardé", "Non_retardé")) %>%
  pivot_wider(names_from = arr_del, values_from = prop)
```

```
print(tab_mois_dplyr)
```

```
## # A tibble: 12 × 3
##   month Non_retardé Retardé
##   <int>      <dbl>    <dbl>
## 1     1         0.713    0.287
## 2     2         0.708    0.292
## 3     3         0.720    0.280
## 4     4         0.665    0.335
## 5     5         0.735    0.265
## 6     6         0.644    0.356
## 7     7         0.635    0.365
## 8     8         0.708    0.292
## 9     9         0.833    0.167
## 10    10         0.779    0.221
## 11    11         0.772    0.228
## 12    12         0.604    0.396
```

```
library(ggplot2)
```

```
# Données au format Long pour ggplot
tab_long <- jn %>%
  filter(!is.na(arr_del)) %>%
  group_by(month, arr_del) %>%
  summarise(N = n(), .groups = "drop") %>%
  group_by(month) %>%
  mutate(prop = N / sum(N)) %>%
  mutate(
    label = ifelse(arr_del, "Retardé", "Non retardé"),
    pourcentage = round(prop * 100, 1)
  )
```

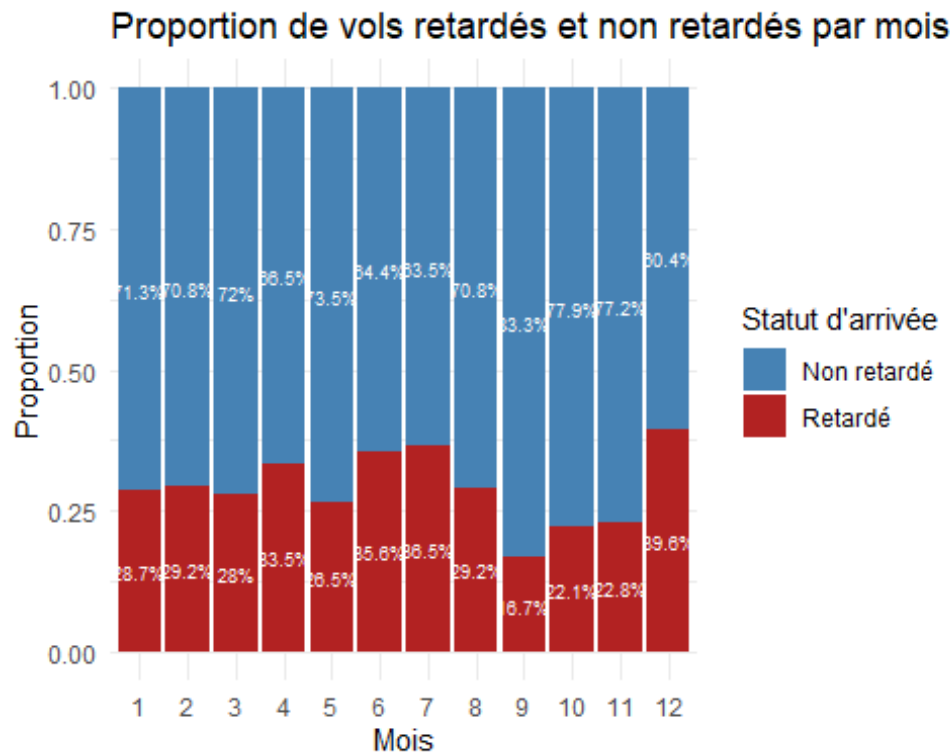
```
# Graphique
```

```
ggplot(tab_long, aes(x = factor(month), y = prop, fill = label)) +
  geom_bar(stat = "identity", position = "stack") +
  geom_text(aes(label = paste0(pourcentage, "%")),
```

```

    position = position_stack(vjust = 0.5), color = "white", size =
2.5) +
  labs(
    x = "Mois", y = "Proportion",
    fill = "Statut d'arrivée",
    title = "Proportion de vols retardés et non retardés par mois"
  ) +
  scale_fill_manual(values = c("Non retardé" = "steelblue", "Retardé" =
"firebrick")) +
  theme_minimal()

```



3.8 Un tableau contenant les retards moyens (en cas de retard) par compagnie (en ligne) × mois (en colonne).

Avec data.table

```

# Calcul : moyenne des retards > 0, par compagnie et mois
retards_dt <- jn_dt[arr_delay > 0, .(
  mean_delay = mean(arr_delay, na.rm = TRUE)
), by = .(carrier, month)]

# Tableau croisé : compagnies en ligne, mois en colonne
retard_tab_dt <- dcast(retards_dt, carrier ~ month, value.var = "mean_delay")
print(retard_tab_dt)

## Key: <carrier>
##   carrier      1      2      3      4      5      6

```

```

7
##      <char>      <num>      <num>      <num>      <num>      <num>      <num>
<num>
##  1:      9E  45.75360 45.77407 47.36917 45.81942 59.11213 67.50166
69.20489
##  2:      AA  29.98693 31.51873 38.57001 46.44093 46.96957 53.12004
46.26927
##  3:      AS  32.17647 17.00000 46.46667 37.43478 39.66667 53.25000
30.23077
##  4:      B6  31.24000 34.44877 37.76699 41.66128 39.07583 54.96138
55.29802
##  5:      DL  27.69671 28.59363 38.02386 36.82620 39.66508 52.23590
56.11045
##  6:      EV  49.64259 43.52297 54.01747 50.36713 44.53154 56.71077
58.65619
##  7:      F9  37.07895 73.44000 44.92000 67.72727 53.47222 48.69444
61.05263
##  8:      FL  21.93878 21.00000 40.00000 34.54315 43.01136 61.89655
67.85714
##  9:      HA 216.00000 74.25000 17.28571 19.60000 16.33333 21.64286
19.00000
## 10:      MQ  31.22993 32.30549 33.83184 44.29936 43.55100 53.53414
49.45560
## 11:      OO 107.00000      NA      NA      NA      NA 157.00000
NA
## 12:      UA  28.15735 26.37571 36.67126 40.34920 38.38841 48.46878
49.43208
## 13:      US  23.79664 21.20708 25.06007 29.91870 27.19135 43.89873
37.46497
## 14:      VX  18.40741 30.57143 35.43038 43.59877 45.68862 64.50403
83.50746
## 15:      WN  29.28899 31.85862 38.89381 46.38149 40.60632 55.57803
54.56494
## 16:      YV  39.80000 38.50000 41.00000 65.50000 39.68750 70.55556
55.09524
##      8      9      10      11      12
##      <num>      <num>      <num>      <num>      <num>
##  1: 46.23062 37.24457 34.74621 33.20611 48.66853
##  2: 37.55093 39.80072 25.19976 26.27368 36.20765
##  3: 50.87500 13.50000 41.00000 20.69231 28.64516
##  4: 41.71258 39.36255 24.67001 26.02764 38.22000
##  5: 36.00647 38.25766 23.90392 23.50657 34.09546
##  6: 45.93747 41.71130 39.52427 32.65986 55.10650
##  7: 44.58065 28.81818 30.32258 36.73684 43.53846
##  8: 41.76608 44.39167 29.49359 35.70866 44.05369
##  9: 21.31250 10.66667 8.50000 19.41667 11.37500
## 10: 36.62770 30.79921 26.14301 25.27333 37.82584
## 11: 64.50000 27.00000      NA 15.00000      NA
## 12: 36.46493 41.34312 26.54272 27.31676 35.39934
## 13: 29.74471 33.40533 21.74910 19.30824 29.41471

```



```
## 14: 34.95758 39.66327 24.00000 30.43523 25.51327
## 15: 43.98765 44.27564 29.77137 28.94050 40.30864
## 16: 52.33333 39.53846 65.17391 40.44444 37.95000
```

Avec dplyr

```
library(tidyr)
retard_tab_dplyr <- jn %>%
  filter(arr_delay > 0) %>%
  group_by(carrier, month) %>%
  summarise(mean_delay = mean(arr_delay, na.rm = TRUE), .groups = "drop") %>%
  pivot_wider(
    names_from = month,
    values_from = mean_delay,
    names_sort = TRUE
  )
```

```
print(retard_tab_dplyr)
```

```
## # A tibble: 16 × 13
##   carrier `1` `2` `3` `4` `5` `6` `7` `8` `9` `10`
##   <chr>   <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1 9E      45.8  45.8  47.4  45.8  59.1  67.5  69.2  46.2  37.2  34.7
## 2 AA      30.0  31.5  38.6  46.4  47.0  53.1  46.3  37.6  39.8  25.2
## 3 AS      32.2  17    46.5  37.4  39.7  53.2  30.2  50.9  13.5  41
## 4 B6      31.2  34.4  37.8  41.7  39.1  55.0  55.3  41.7  39.4  24.7
## 5 DL      27.7  28.6  38.0  36.8  39.7  52.2  56.1  36.0  38.3  23.9
## 6 EV      49.6  43.5  54.0  50.4  44.5  56.7  58.7  45.9  41.7  39.5
## 7 F9      37.1  73.4  44.9  67.7  53.5  48.7  61.1  44.6  28.8  30.3
## 8 FL      21.9  21    40    34.5  43.0  61.9  67.9  41.8  44.4  29.5
## 9 HA      216   74.2  17.3  19.6  16.3  21.6  19    21.3  10.7  8.5
## 10 MQ      31.2  32.3  33.8  44.3  43.6  53.5  49.5  36.6  30.8  26.1
## 11 OO      107   NA    NA    NA    NA    157   NA    64.5  27    NA
## 12 UA      28.2  26.4  36.7  40.3  38.4  48.5  49.4  36.5  41.3  26.5
## 13 US      23.8  21.2  25.1  29.9  27.2  43.9  37.5  29.7  33.4  21.7
## 14 VX      18.4  30.6  35.4  43.6  45.7  64.5  83.5  35.0  39.7  24
```

```
## 15 WN      29.3  31.9  38.9  46.4  40.6  55.6  54.6  44.0  44.3  29.8
28.9
## 16 YV      39.8  38.5  41    65.5  39.7  70.6  55.1  52.3  39.5  65.2
40.4
## # [i] 1 more variable: `12` <dbl>
```

3.9 Pour chaque destination et chaque vol, calculez le retard en vol (time.delay) = temps de vol par rapport au temps de vol le plus court vers cette destination et sauvez cette nouvelle variable dans la table, puis pour chaque destination, donnez le vol qui a eu le plus de retard en vol avec son origine et sa destination.

Avec data.table

```
# 1. Calcul du temps de vol le plus court par destination
jn_dt[, min_air_time := min(air_time, na.rm = TRUE), by = dest]

# 2. Calcul du retard en vol
jn_dt[, time_delay := air_time - min_air_time]

# 3. Sélection du vol le plus retardé en vol pour chaque destination
retard_max_dt <- jn_dt[!is.na(time_delay), .SD[which.max(time_delay)], by =
dest]

# Affichage des colonnes utiles
retard_max_dt[, .(dest, origin, flight, time_delay)]

##      dest origin flight time_delay
##      <char> <char>  <int>      <num>
##  1:   MSP    LGA   3588         120
##  2:   IAD    JFK   5711          61
##  3:   BUF    JFK   2906          67
##  4:   SYR    JFK   1516          67
##  5:   ROC    JFK    686          60
##  ---
## 100:   BZN    EWR    336          46
## 101:   MTJ    EWR    486          57
## 102:   ANC    EWR    887          46
## 103:   PSP    JFK     55          64
## 104:   MDW    LGA     3          92
```

Avec dplyr

```
# 1 & 2. Calcul du temps de vol minimal et du retard en vol
jn <- jn %>%
  group_by(dest) %>%
  mutate(min_air_time = min(air_time, na.rm = TRUE)) %>%
  ungroup() %>%
  mutate(time_delay = air_time - min_air_time)
```

```
## Warning: There was 1 warning in `mutate()`.
## [i] In argument: `min_air_time = min(air_time, na.rm = TRUE)`.
## [i] In group 52: `dest = "LGA"`.
## Caused by warning in `min()`:
## ! aucun argument trouvé pour min ; Inf est renvoyé

# 3. Vol le plus en retard en vol par destination
retard_max_dplyr <- jn %>%
  filter(!is.na(time_delay)) %>%
  group_by(dest) %>%
  slice_max(time_delay, n = 1, with_ties = FALSE) %>%
  select(dest, origin, flight, time_delay)

print(retard_max_dplyr)

## # A tibble: 104 × 4
## # Groups:   dest [104]
##   dest origin flight time_delay
##   <chr> <chr>   <int>     <dbl>
## 1 ABQ   JFK       65       106
## 2 ACK   JFK     1491       106
## 3 ALB   EWR     4117        26
## 4 ANC   EWR      887        46
## 5 ATL   EWR      926       111
## 6 AUS   JFK     1295       127
## 7 AVL   EWR     4175        43
## 8 BDL   EWR     4276        36
## 9 BGR   LGA     5207        21
## 10 BHM  LGA     5038        61
## # [i] 94 more rows
```

3.10 Donner par mois la destination la plus retardée à l'arrivée. On affichera son retard moyen à l'arrivée et au départ, le nombre et le pourcentage de vols retardés à l'arrivée, la vitesse moyenne de vol en km, la distance moyenne parcourue en km ainsi que le retard en vol moyen (cf 9)).

Avec data.table

```
# 1. Calcul des indicateurs par mois et destination
retard_mois_dest_dt <- jn_dt[!is.na(arr_delay), .(
  mean_arr_delay = mean(arr_delay, na.rm = TRUE),
  mean_dep_delay = mean(dep_delay, na.rm = TRUE),
  nb_retards_arr = sum(arr_del == TRUE, na.rm = TRUE),
  total_vols = .N,
  pct_retards_arr = mean(arr_del == TRUE, na.rm = TRUE),
  mean_vit_km = mean(vit_km, na.rm = TRUE),
  mean_dist_km = mean(dist_km, na.rm = TRUE),
  mean_time_delay = mean(time_delay, na.rm = TRUE)
), by = .(month, dest)]
```

2. Sélection de la destination la plus retardée à l'arrivée par mois

```
retard_max_mois_dt <- retard_mois_dest_dt[
  , .SD[which.max(mean_arr_delay)], by = month
]
```

Affichage

```
retard_max_mois_dt
```

##	month	dest	mean_arr_delay	mean_dep_delay	nb_retards_arr	total_vols	
##	<int>	<char>	<num>	<num>	<int>	<int>	
##	1:	1	TUL	68.11538	56.50000	19	26
##	2:	10	CAE	58.55556	50.11111	7	9
##	3:	11	SBN	51.50000	67.50000	2	2
##	4:	12	BZN	69.25000	75.00000	3	4
##	5:	2	DSM	48.18182	51.22727	16	22
##	6:	3	DSM	60.57692	71.03846	14	26
##	7:	4	CAE	71.33333	41.50000	5	6
##	8:	5	OKC	38.32143	42.46429	17	28
##	9:	6	CAE	54.55556	33.77778	8	9
##	10:	7	CAE	92.14286	69.42857	6	7
##	11:	8	CAE	50.75000	45.37500	7	8
##	12:	9	CAE	21.37500	12.12500	5	8
##	pct_retards_arr	mean_vit_km	mean_dist_km	mean_time_delay			
##	<num>	<num>	<num>	<num>			
##	1:	0.7307692	592.5208	1955.3481	51.692308		
##	2:	0.7777778	620.6174	968.8227	15.111111		
##	3:	1.0000000	606.4495	1047.6803	16.000000		
##	4:	0.7500000	678.8920	3028.7779	26.750000		
##	5:	0.7272727	630.9418	1636.6988	33.181818		
##	6:	0.5384615	658.5945	1636.6988	26.500000		
##	7:	0.8333333	607.4726	968.8227	17.166667		
##	8:	0.6071429	708.8826	2132.3755	19.035714		
##	9:	0.8888889	617.7606	968.8227	15.666667		
##	10:	0.8571429	659.4880	972.2713	9.857143		
##	11:	0.8750000	678.3345	968.8227	7.000000		
##	12:	0.6250000	674.2999	968.8227	7.625000		

Avec dplyr

```
library(dplyr)
```

1. Calcul des indicateurs par mois et destination

```
retard_mois_dest_dplyr <- jn %>%
  filter(!is.na(arr_delay)) %>%
  group_by(month, dest) %>%
  summarise(
    mean_arr_delay = mean(arr_delay, na.rm = TRUE),
    mean_dep_delay = mean(dep_delay, na.rm = TRUE),
    nb_retards_arr = sum(arr_del == TRUE, na.rm = TRUE),
```

```

    total_vols = n(),
    pct_retards_arr = mean(arr_del == TRUE, na.rm = TRUE),
    mean_vit_km = mean(vit_km, na.rm = TRUE),
    mean_dist_km = mean(dist_km, na.rm = TRUE),
    mean_time_delay = mean(time_delay, na.rm = TRUE),
    .groups = "drop"
  )
}

# 2. Sélection de la destination la plus retardée à l'arrivée par mois
retard_max_mois_dplyr <- retard_mois_dest_dplyr %>%
  group_by(month) %>%
  slice_max(mean_arr_delay, n = 1, with_ties = FALSE) %>%
  ungroup()

# Affichage
print(retard_max_mois_dplyr)

## # A tibble: 12 × 10
##   month dest mean_arr_delay mean_dep_delay nb_retards_arr total_vols
##   <int> <chr>          <dbl>          <dbl>          <int>      <int>
## 1     1  TUL           68.1           56.5            19         26
## 2     2  DSM           48.2           51.2            16         22
## 3     3  DSM           60.6           71.0            14         26
## 4     4  CAE           71.3           41.5             5          6
## 5     5  OKC           38.3           42.5            17         28
## 6     6  CAE           54.6           33.8             8          9
## 7     7  CAE           92.1           69.4             6          7
## 8     8  CAE           50.8           45.4             7          8
## 9     9  CAE           21.4           12.1             5          8
## 10    10  CAE           58.6           50.1             7          9
## 11    11  SBN           51.5           67.5             2          2
## 12    12  BZN           69.2           75              3          4
## # [i] 4 more variables: pct_retards_arr <dbl>, mean_vit_km <dbl>,
## #   mean_dist_km <dbl>, mean_time_delay <dbl>

```

3.11 Un graphique permettant de montrer comment le retard moyen à l'arrivée varie en fonction du mois et de la destination.

Avec data.table

```

# Calcul du retard moyen à l'arrivée par mois et destination
retard_arrivee_mois_dest_dt <- jn_dt[!is.na(arr_delay),
  .(mean_arr_delay = mean(arr_delay, na.rm
= TRUE)),
  by = .(month, dest)]

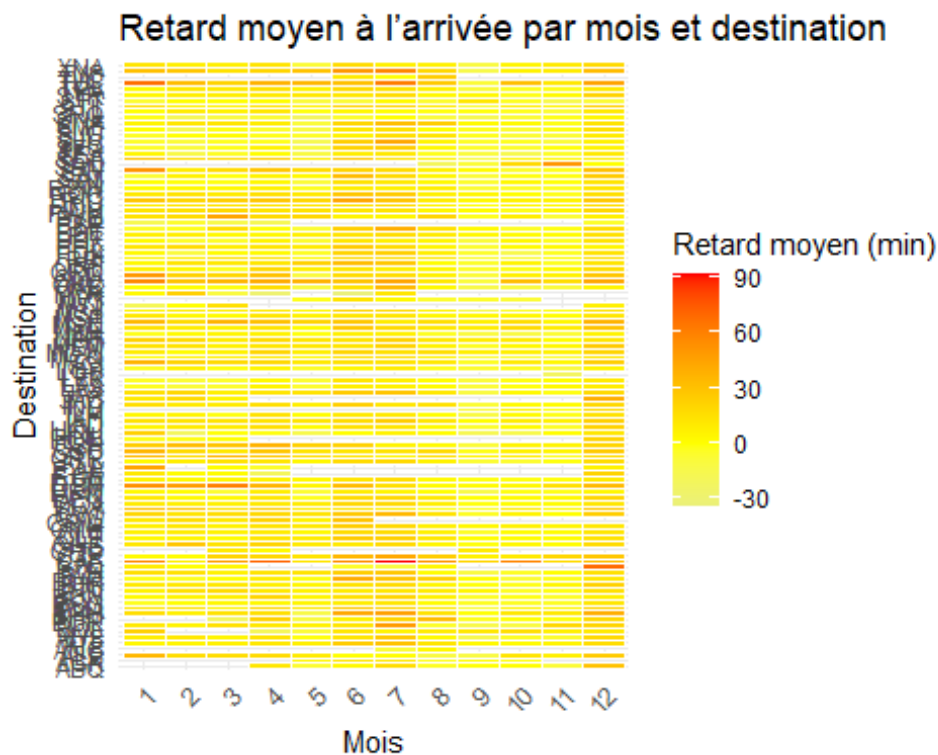
# Graphique heatmap
ggplot(retard_arrivee_mois_dest_dt, aes(x = factor(month), y = dest, fill =
mean_arr_delay)) +
  geom_tile(color = "white") +

```

```

scale_fill_gradient2(
  low = "lightblue",
  mid = "yellow",
  high = "red",
  midpoint = 0,
  name = "Retard moyen (min)"
) +
labs(
  x = "Mois",
  y = "Destination",
  title = "Retard moyen à l'arrivée par mois et destination"
) +
theme_minimal() +
theme(axis.text.x = element_text(angle = 45, hjust = 1))

```



Avec dplyr

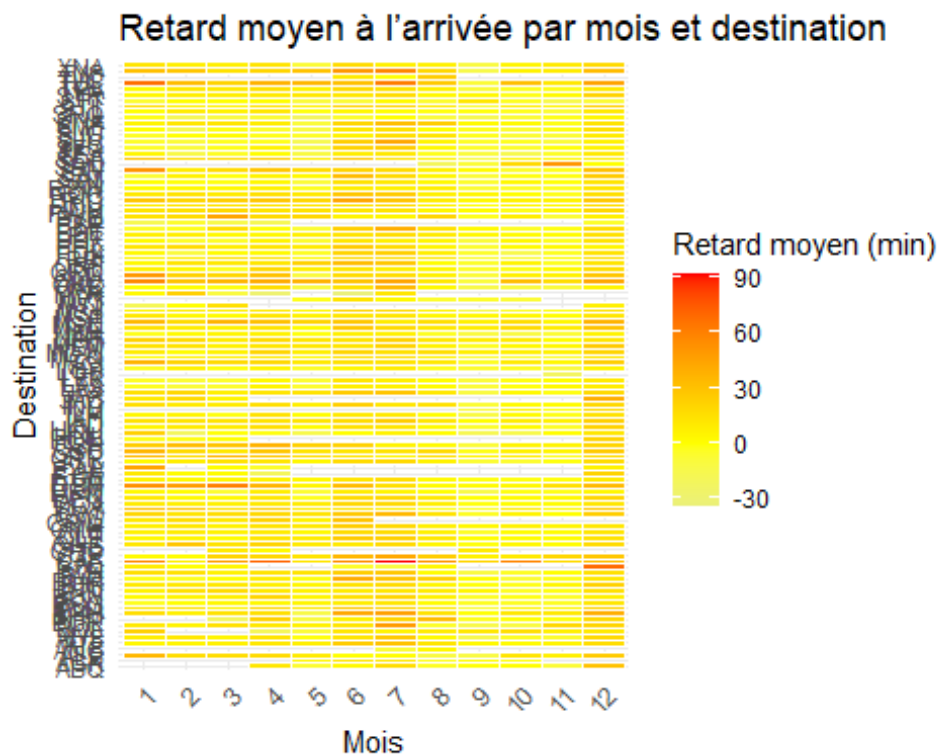
```

library(dplyr)
library(ggplot2)

# Calcul du retard moyen à l'arrivée par mois et destination
retard_arrivee_mois_dest <- jn %>%
  filter(!is.na(arr_delay)) %>%
  group_by(month, dest) %>%
  summarise(mean_arr_delay = mean(arr_delay, na.rm = TRUE), .groups = "drop")
ggplot(retard_arrivee_mois_dest, aes(x = factor(month), y = dest, fill =
mean_arr_delay)) +

```

```
geom_tile(color = "white") +
scale_fill_gradient2(
  low = "lightblue",
  mid = "yellow",
  high = "red",
  midpoint = 0,
  name = "Retard moyen (min)"
) +
labs(
  x = "Mois",
  y = "Destination",
  title = "Retard moyen à l'arrivée par mois et destination"
) +
theme_minimal() +
theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



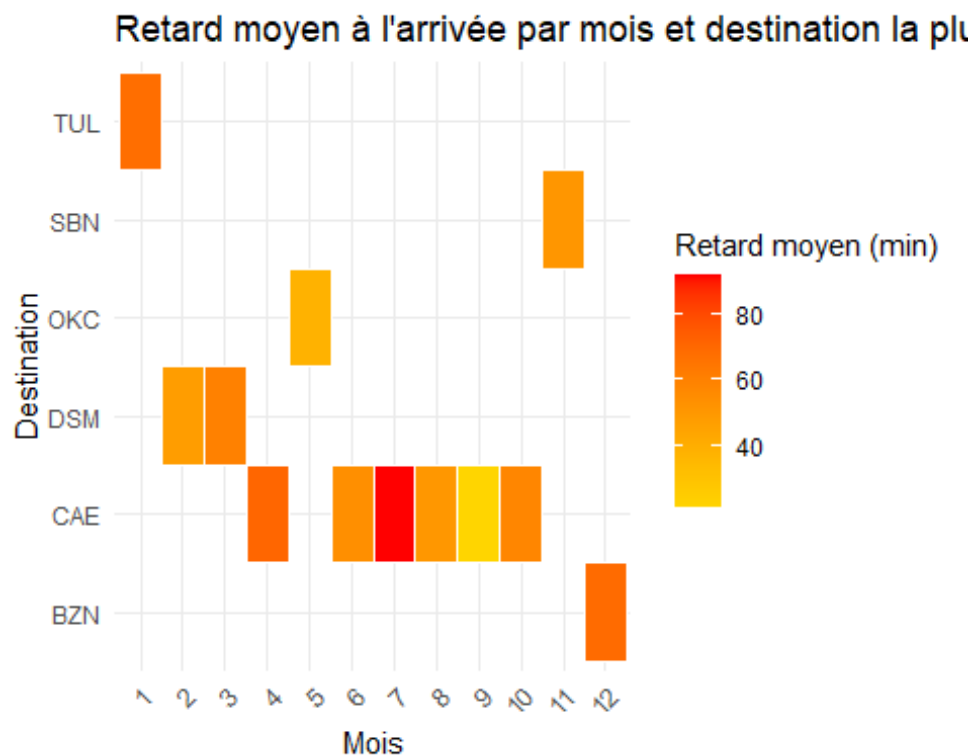
```
library(ggplot2)

# Graphique pour visualiser Le retard moyen à l'arrivée par mois et
# destination
# par mois la destination la plus retardée à l'arrivée
ggplot(retard_max_mois_dplyr, aes(x = factor(month), y = dest, fill =
mean_arr_delay)) +
  geom_tile(color = "white") +
  scale_fill_gradient2(
    low = "lightblue",      # Retards en avance (négatifs)
    mid = "yellow",         # Retards modérés
```

```

    high = "red",                # Plus grands retards
    midpoint = 0,
    name = "Retard moyen (min)"
  ) +
  labs(
    x = "Mois",
    y = "Destination",
    title = "Retard moyen à l'arrivée par mois et destination la plus
retardée"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))

```



3.12 Pour un certain nombre de vols, la valeur du retard à l'arrivée est manquante. Ces valeurs manquantes sont probablement des vols annulés. Calculer et représenter le nombre de vols annulés par jour. Y a-t-il une tendance ?

Avec data.table

```

library(data.table)

# Compter les vols annulés par jour
annules_jour_dt <- jn_dt[is.na(arr_delay),
                        .(nb_annules = .N),
                        by = .(year, month, day)]

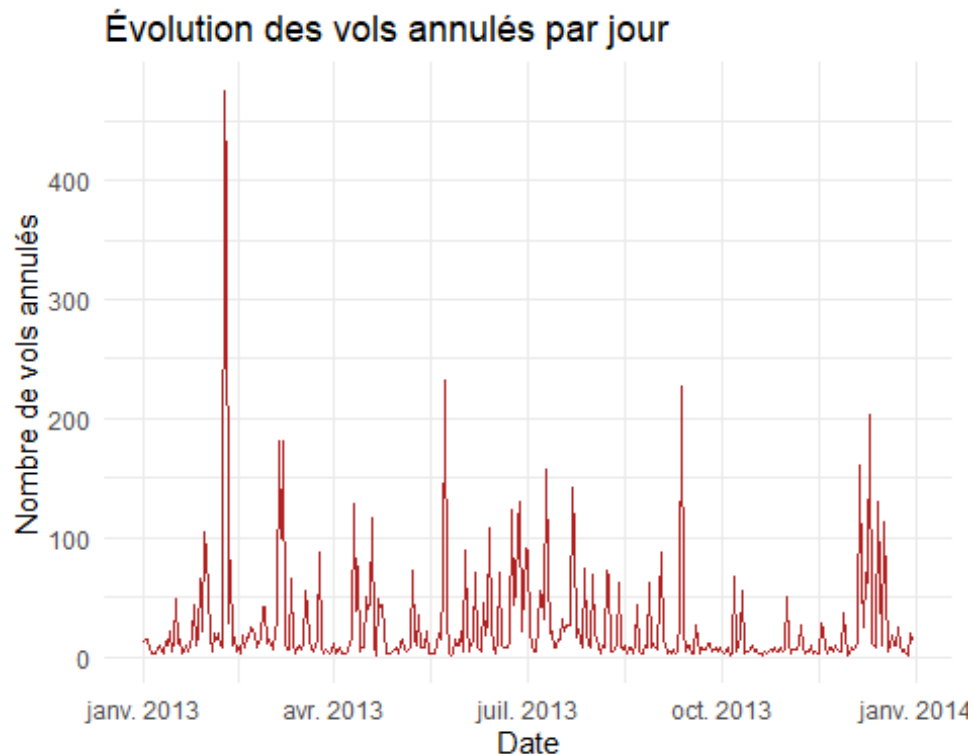
```



```
annules_jour_dt[, date := as.Date(sprintf("%04d-%02d-%02d", year, month,
day))]
```

Graphique

```
ggplot(annules_jour_dt, aes(x = date, y = nb_annules)) +
  geom_line(color = "firebrick") +
  labs(
    x = "Date",
    y = "Nombre de vols annulés",
    title = "Évolution des vols annulés par jour"
  ) +
  theme_minimal()
```



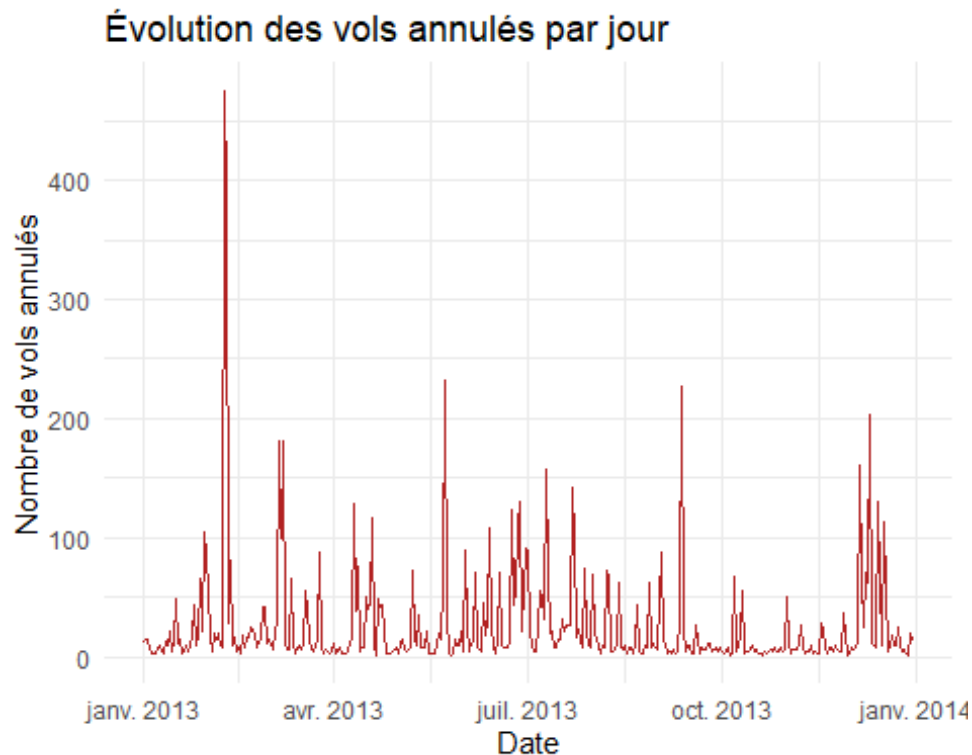
Avec dplyr

```
library(dplyr)
library(ggplot2)

# Identifier les vols annulés (arr_delay manquant) et compter par jour
annules_jour <- jn %>%
  filter(is.na(arr_delay)) %>%
  group_by(year, month, day) %>%
  summarise(nb_annules = n(), .groups = "drop") %>%
  mutate(date = as.Date(sprintf("%04d-%02d-%02d", year, month, day)))

# Affichage du graphique
ggplot(annules_jour, aes(x = date, y = nb_annules)) +
```

```
geom_line(color = "firebrick") +
labs(
  x = "Date",
  y = "Nombre de vols annulés",
  title = "Évolution des vols annulés par jour"
) +
theme_minimal()
```



Entre janvier et avril 2013, on observe un pic assez important, plusieurs pics de vols annulés, probablement dus à des conditions météorologiques hivernales. D'autres hausses ponctuelles apparaissent à divers moments de l'année, liées à des événements isolés. Globalement, aucune tendance claire ne se dégage sur l'ensemble de l'année.

3.13 On utilise la table `planes` pour avoir des informations sur les avions utilisés. Joindre la table principale à la table `planes` pour inclure l'année de fabrication (ancienneté) de l'avion. A l'aide de tableaux et/ou graphiques, répondez aux questions suivantes : existe-t-il une relation

- Entre le retard au départ et le retard à l'arrivée ?
- Entre l'année de fabrication de l'avion et son retard à l'arrivée ?
- Entre le retard en vol et le retard à l'arrivée ?
- Entre la destination et le retard à l'arrivée ? (On pourra se limiter à une seule syntaxe, entre `dplyr` et `data.table`)

```
# Jointure pour ajouter l'année de fabrication
jn_planes <- jn %>%
```

```

left_join(planes %>% select(tailnum, plane_year = year), by = "tailnum")
head(jn_planes)

## # A tibble: 6 × 27
##   year month   day dep_time sched_dep_time dep_delay arr_time
##   <int> <int> <int>   <int>         <int>       <dbl>   <int>
##   <int>
## 1  2013     1     1     517           515         2     830
819
## 2  2013     1     1     533           529         4     850
830
## 3  2013     1     1     542           540         2     923
850
## 4  2013     1     1     544           545        -1    1004
1022
## 5  2013     1     1     554           600        -6     812
837
## 6  2013     1     1     554           558        -4     740
728
## # [i] 19 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance
<dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dtm>, name <chr>, dist_km <dbl>,
## #   vit_km <dbl>, arr_del <lgl>, dep_del <lgl>, min_air_time <dbl>,
## #   time_delay <dbl>, plane_year <int>

```

3.13.1 Relation entre le retard au départ et à l'arrivée

```

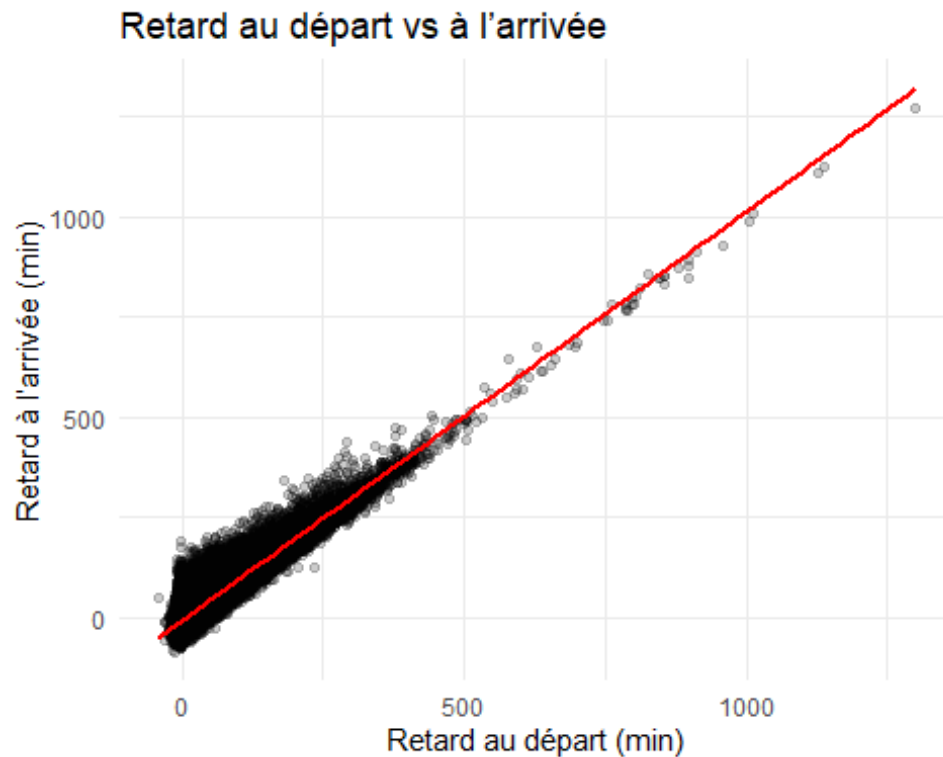
ggplot(jn_planes, aes(x = dep_delay, y = arr_delay)) +
  geom_point(alpha = 0.2) +
  geom_smooth(method = "lm", color = "red") +
  labs(title = "Retard au départ vs à l'arrivée", x = "Retard au départ
(min)", y = "Retard à l'arrivée (min)") +
  theme_minimal()

## `geom_smooth()` using formula = 'y ~ x'

## Warning: Removed 9430 rows containing non-finite outside the scale range
## (`stat_smooth()`).

## Warning: Removed 9430 rows containing missing values or values outside the
scale range
## (`geom_point()`).

```



```
# Calcul de la corrélation entre le retard au départ et à l'arrivée
corr_dep_arr_delay <- cor(jn_planes$dep_delay, jn_planes$arr_delay, use =
"complete.obs")
corr_dep_arr_delay
## [1] 0.9148028
```

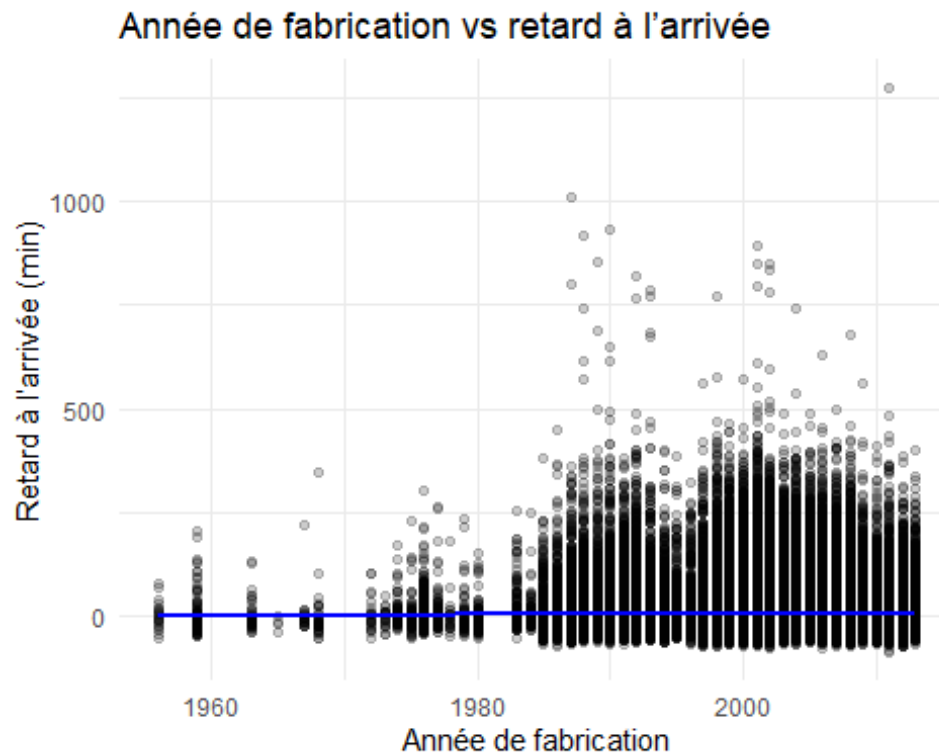
Corrélation : 0.91

Relation très forte : Il existe une corrélation positive très élevée, ce qui signifie que plus un vol part en retard, plus il a de chances d'arriver en retard.

3.13.2 Relation entre l'année de fabrication et le retard à l'arrivée

```
ggplot(jn_planes %>% filter(!is.na(plane_year), !is.na(arr_delay)),
  aes(x = plane_year, y = arr_delay)) +
  geom_point(alpha = 0.2) +
  geom_smooth(method = "lm", color = "blue") +
  labs(title = "Année de fabrication vs retard à l'arrivée", x = "Année de
fabrication", y = "Retard à l'arrivée (min)") +
  theme_minimal()

## `geom_smooth()` using formula = 'y ~ x'
```



```
# Calcul de la corrélation entre L'année de fabrication et Le retard à
L'arrivée
corr_year_arr_delay <- cor(jn_planes$plane_year, jn_planes$arr_delay, use =
"complete.obs")
corr_year_arr_delay
## [1] 0.01767153
```

Corrélation : 0.0177

Aucune relation notable : L'ancienneté de l'avion n'a pratiquement aucun impact sur le retard à l'arrivée.

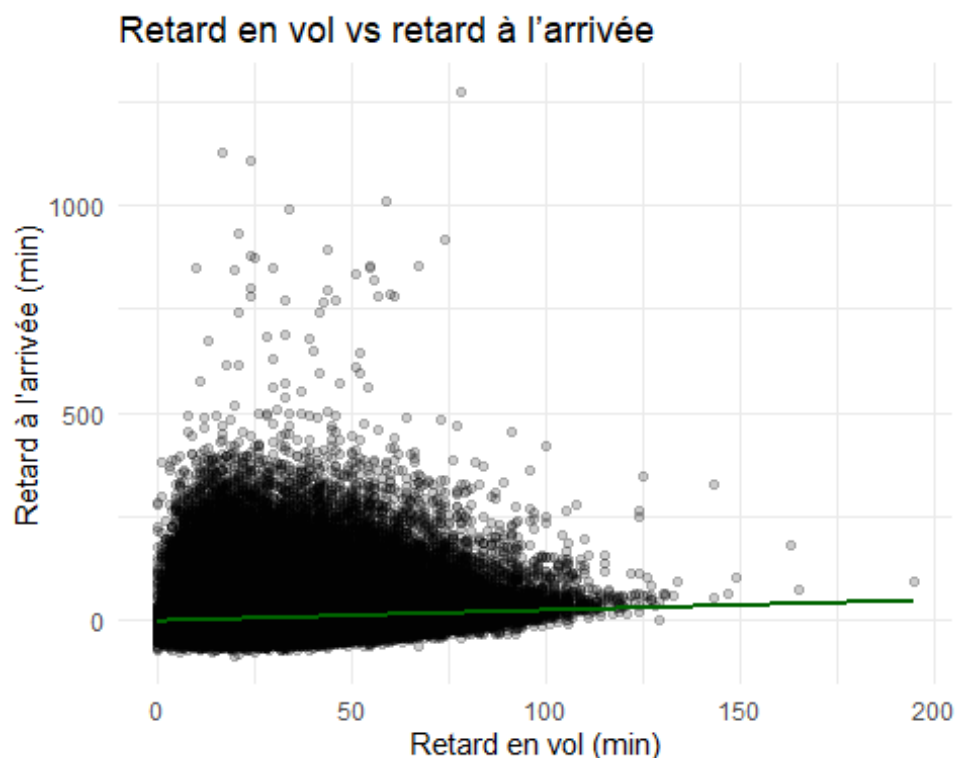
3.13.3 Relation entre le retard en vol (time_delay) et le retard à l'arrivée

```
ggplot(jn_planes, aes(x = time_delay, y = arr_delay)) +
  geom_point(alpha = 0.2) +
  geom_smooth(method = "lm", color = "darkgreen") +
  labs(title = "Retard en vol vs retard à l'arrivée", x = "Retard en vol
(min)", y = "Retard à l'arrivée (min)") +
  theme_minimal()

## `geom_smooth()` using formula = 'y ~ x'

## Warning: Removed 9430 rows containing non-finite outside the scale range
## (`stat_smooth()`).
```

```
## Warning: Removed 9430 rows containing missing values or values outside the
scale range
## (`geom_point()`).
```



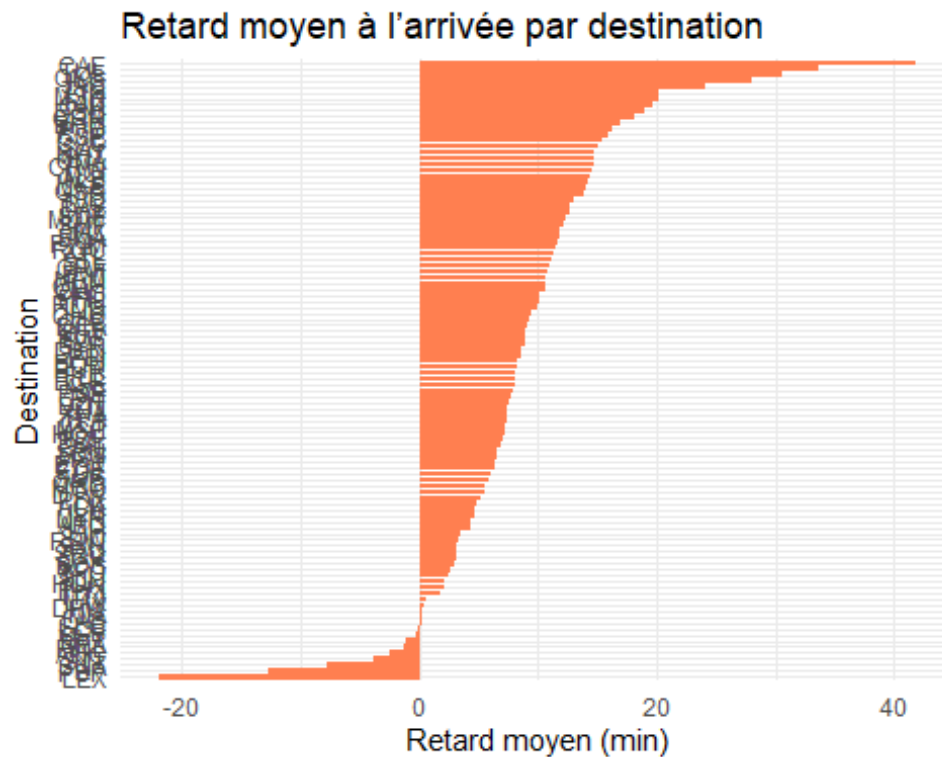
```
# Calcul de la corrélation entre le retard en vol et le retard à l'arrivée
corr_time_delay_arr_delay <- cor(jn_planes$time_delay, jn_planes$arr_delay,
use = "complete.obs")
corr_time_delay_arr_delay
## [1] 0.09980908
```

Corrélation : 0.0998.

Relation très faible : Il y a un lien léger, mais pas suffisamment fort pour affirmer que le retard en vol explique à lui seul le retard à l'arrivée.

3.13.4 Relation entre la destination et le retard à l'arrivée

```
jn_planes %>%
  filter(!is.na(arr_delay)) %>%
  group_by(dest) %>%
  summarise(mean_arr_delay = mean(arr_delay, na.rm = TRUE)) %>%
  ggplot(aes(x = reorder(dest, mean_arr_delay), y = mean_arr_delay)) +
  geom_col(fill = "coral") +
  coord_flip() +
  labs(title = "Retard moyen à l'arrivée par destination", x = "Destination",
y = "Retard moyen (min)") +
  theme_minimal()
```



```
# Calcul du retard moyen à l'arrivée par destination
retard_dest <- jn_planes %>%
  group_by(dest) %>%
  summarise(mean_arr_delay = mean(arr_delay, na.rm = TRUE))

# Affichage du tableau
head(retard_dest)

## # A tibble: 6 × 2
##   dest mean_arr_delay
##   <chr>         <dbl>
## 1 ABQ             4.38
## 2 ACK             4.85
## 3 ALB            14.4
## 4 ANC            -2.5
## 5 ATL            11.3
## 6 AUS             6.02
```

On observe des différences importantes selon les destinations. Certaines destinations comme CAE, TUL, OKC ont des retards moyens élevés (> 30 min), tandis que d'autres comme LAX, SEA, HNL ont des retards très faibles, voire négatifs (vols arrivés en avance). La destination influence clairement le retard moyen à l'arrivée.

La différences de retard moyen selon les destinations, suggère une certaine influence de la destination sur les retards à l'arrivée. Toutefois, pour comprendre cette relation en profondeur, il serait nécessaire de considérer d'autres paramètres explicatifs comme l'origine du vol, l'année de fabrication du modèle de l'avion, les conditions météorologiques

locales ou encore la densité du trafic aérien. De plus, des méthodes statistiques seraient utiles pour mieux isoler l'effet propre de la destination.

3.14 On souhaite expliquer le retard à l'arrivée par les conditions météorologiques au départ. Effectuer une jointure de la table principale à la table weather et calculer la température en degrés Celsius (on rappelle que $t^F = 95t^C + 32$.)

data.table

```
library(data.table)

# Copier l'objet weather pour le modifier localement
weather_copy <- copy(weather)

# Convertir en data.table
setDT(weather_copy)

# S'assurer que jn est aussi un data.table
setDT(jn)

# Faire la jointure
jn_weather_dt <- merge(
  jn,
  weather_copy[, .(origin, year, month, day, hour, temp, humid, wind_speed,
    precip)],
  by = c("origin", "year", "month", "day", "hour"),
  all.x = TRUE
)

# Calcul de la température en Celsius
jn_weather_dt[, temp_c := (temp - 32) * 5 / 9]
head(jn_weather_dt)

## Key: <origin, year, month, day, hour>
##   origin year month  day  hour dep_time sched_dep_time dep_delay
##   <char> <int> <int> <int> <int>    <int>         <int>      <num>
##   <int>
## 1:   EWR  2013     1     1     5     517           515          2
## 830
## 2:   EWR  2013     1     1     5     554           558         -4
## 740
## 3:   EWR  2013     1     1     6     555           600         -5
## 913
## 4:   EWR  2013     1     1     6     558           600         -2
## 923
## 5:   EWR  2013     1     1     6     559           600         -1
## 854
```



```
## 6:      EWR  2013      1      1      6      601      600      1
844
##      sched_arr_time arr_delay carrier flight tailnum  dest air_time
distance
##              <int>      <num> <char> <int> <char> <char>      <num>
<num>
## 1:              819          11      UA   1545 N14228   IAH       227
1400
## 2:              728          12      UA   1696 N39463   ORD       150
719
## 3:              854          19      B6    507 N516JB   FLL       158
1065
## 4:              937         -14      UA   1124 N53441   SFO       361
2565
## 5:              902          -8      UA   1187 N76515   LAS       337
2227
## 6:              850          -6      B6    343 N644JB   PBI       147
1023
##      minute          time_hour          name  dist_km  vit_km
arr_del
##      <num>          <POS<          <char>      <num>      <num>
<lgcl>
## 1:      15 2013-01-01 05:00:00 United Air Lines Inc. 2253.076 595.5267
TRUE
## 2:      58 2013-01-01 05:00:00 United Air Lines Inc. 1157.115 462.8462
TRUE
## 3:       0 2013-01-01 06:00:00      JetBlue Airways 1713.947 650.8660
TRUE
## 4:       0 2013-01-01 06:00:00 United Air Lines Inc. 4127.957 686.0871
FALSE
## 5:       0 2013-01-01 06:00:00 United Air Lines Inc. 3584.000 638.1009
FALSE
## 6:       0 2013-01-01 06:00:00      JetBlue Airways 1646.355 671.9816
FALSE
##      dep_del min_air_time time_delay  temp humid wind_speed precip temp_c
##      <lgcl>      <num>      <num> <num> <num>      <num>      <num>      <num>
## 1:  FALSE      161          66 39.02 64.43   12.65858      0      3.9
## 2:  FALSE       87          63 39.02 64.43   12.65858      0      3.9
## 3:  FALSE      123          35 37.94 67.21   11.50780      0      3.3
## 4:  FALSE      295          66 37.94 67.21   11.50780      0      3.3
## 5:  FALSE      256          81 37.94 67.21   11.50780      0      3.3
## 6:  FALSE      105          42 37.94 67.21   11.50780      0      3.3
```

Avec dplyr

```
library(dplyr)
```

```
# 1. Jointure avec la table weather (attention aux colonnes communes : year,
month, day, hour, origin)
jn_weather <- jn %>%
```

```

left_join(weather %>%
  select(origin, year, month, day, hour, temp, humid, wind_speed,
precip),
  by = c("origin", "year", "month", "day", "hour"))

```

2. Conversion de La température de Fahrenheit vers Celsius

```

jn_weather <- jn_weather %>%
  mutate(temp_c = (temp - 32) * 5 / 9)

```

3. Afficher un aperçu

```
head(jn_weather)
```

```

##      year month   day dep_time sched_dep_time dep_delay arr_time
##      <int> <int> <int>   <int>           <int>      <num>   <int>
##      <int>
## 1:  2013     1     1     517             515         2     830
819
## 2:  2013     1     1     533             529         4     850
830
## 3:  2013     1     1     542             540         2     923
850
## 4:  2013     1     1     544             545        -1    1004
1022
## 5:  2013     1     1     554             600        -6     812
837
## 6:  2013     1     1     554             558        -4     740
728
##      arr_delay carrier flight tailnum origin  dest air_time distance  hour
##      <num>   <char> <int>   <char> <char> <char>   <num>   <num> <num>
## 1:         11     UA   1545  N14228   EWR   IAH     227    1400    5
## 2:         20     UA   1714  N24211   LGA   IAH     227    1416    5
## 3:         33     AA   1141  N619AA   JFK   MIA     160    1089    5
## 4:        -18     B6    725  N804JB   JFK   BQN     183    1576    5
## 5:        -25     DL    461  N668DN   LGA   ATL     116     762    6
## 6:         12     UA   1696  N39463   EWR   ORD     150     719    5
##      minute      time_hour      name  dist_km  vit_km
##      <num>           <POSc>      <char>   <num>   <num>
##      <lgcl>
## 1:      15 2013-01-01 05:00:00 United Air Lines Inc. 2253.076 595.5267
TRUE
## 2:      29 2013-01-01 05:00:00 United Air Lines Inc. 2278.825 602.3327
TRUE
## 3:      40 2013-01-01 05:00:00 American Airlines Inc. 1752.571 657.2142
TRUE
## 4:      45 2013-01-01 05:00:00      JetBlue Airways 2536.320 831.5803
FALSE
## 5:       0 2013-01-01 06:00:00 Delta Air Lines Inc. 1226.317 634.3019
FALSE

```

```
## 6:      58 2013-01-01 05:00:00 United Air Lines Inc. 1157.115 462.8462
TRUE
##      dep_del min_air_time time_delay  temp humid wind_speed precip temp_c
##      <lgcl>      <num>      <num> <num> <num>      <num> <num> <num>
## 1:    FALSE      161      66 39.02 64.43  12.65858      0    3.9
## 2:    FALSE      161      66 39.92 54.81  14.96014      0    4.4
## 3:    FALSE      125      35 39.02 61.63  14.96014      0    3.9
## 4:    FALSE      173      10 39.02 61.63  14.96014      0    3.9
## 5:    FALSE       65      51 39.92 54.81  16.11092      0    4.4
## 6:    FALSE       87      63 39.02 64.43  12.65858      0    3.9
```

3.15 Le lien entre les variables météo au départ et le retard à l'arrivée. On pourra effectuer par exemple une régression linéaire expliquant le retard à l'arrivée, ou une régression logistique expliquant l'indicatrice de retard à l'arrivée (On pourra se limiter à une seule syntaxe entre dplyr et data.table).

```
# Filtrage pour la régression logistique
data_reg_log <- jn_weather_dt %>%
  filter(!is.na(arr_del), !is.na(temp_c), !is.na(humid), !is.na(wind_speed),
    !is.na(precip))

# Régression logistique
modele_logit <- glm(arr_del ~ temp_c + humid + wind_speed + precip,
  data = data_reg_log,
  family = binomial)

# Résumé du modèle
summary(modele_logit)

##
## Call:
## glm(formula = arr_del ~ temp_c + humid + wind_speed + precip,
##      family = binomial, data = data_reg_log)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -2.2301730  0.0184724 -120.73  <2e-16 ***
## temp_c      0.0043945  0.0004005   10.97  <2e-16 ***
## humid       0.0137154  0.0002129   64.43  <2e-16 ***
## wind_speed  0.0379827  0.0007293   52.08  <2e-16 ***
## precip      4.6452120  0.1593550   29.15  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 392238  on 325723  degrees of freedom
## Residual deviance: 383527  on 325719  degrees of freedom
## AIC: 383537
##
## Number of Fisher Scoring iterations: 4
```

```
# odds ratios
exp(coef(modele_logit))

## (Intercept)      temp_c      humid  wind_speed      precip
##  0.1075098    1.0044041    1.0138099    1.0387132  104.0854323

# Pseudo R² de McFadden
null_dev <- modele_logit$null.deviance
resid_dev <- modele_logit$deviance
pseudo_R2 <- 1 - (resid_dev / null_dev)
cat("Pseudo R² de McFadden :", round(pseudo_R2, 4), "\n")

## Pseudo R² de McFadden : 0.0222
```

Toutes les variables météo sont hautement significatives ($p < 2e-16$). La variable précipitation a un effet très fort : elle multiplie par plus de 100 les chances de retard lorsqu'elle augmente d'une unité. Bien que les effets de temp_c, humid, et wind_speed soient plus modestes individuellement, ils contribuent significativement à l'augmentation de la probabilité de retard.

Le pseudo R² de McFadden (0.0222) est très faible $\Rightarrow$ le modèle n'explique qu'une faible part du phénomène.

3.16 Les retards sont généralement corrélés dans le temps : même une fois que le problème à l'origine du retard initial a été résolu, les vols ultérieurs sont retardés pour permettre aux vols antérieurs de partir. En utilisant la fonction lag(), étudiez comment le retard d'un vol est lié au retard du vol précédent (à ne faire qu'avec dplyr).

```
library(dplyr)

# Nettoyage de base : garder les colonnes essentielles
data_lag <- jn_planes %>%
  filter(!is.na(tailnum), !is.na(dep_delay)) %>%
  arrange(tailnum, year, month, day, sched_dep_time) # Trier par avion et
  par ordre chronologique

# Ajouter la variable lag : retard du vol précédent pour le même avion
data_lag <- data_lag %>%
  group_by(tailnum) %>%
  mutate(prev_dep_delay = lag(dep_delay)) %>%
  ungroup()

# Filtrer les lignes avec un retard précédent non manquant
data_lag_clean <- data_lag %>%
  filter(!is.na(prev_dep_delay))

# Calcul de la corrélation entre les deux retards
corr_retards <- cor(data_lag_clean$dep_delay, data_lag_clean$prev_dep_delay,
  use = "complete.obs")
```

```

cat("Corrélation entre le retard d'un vol et celui du vol précédent (même
avion) :", round(corr_retards, 4), "\n")

## Corrélation entre le retard d'un vol et celui du vol précédent (même
avion) : 0.1011

# Régression linéaire simple pour modéliser la relation
modele_lag <- lm(dep_delay ~ prev_dep_delay, data = data_lag_clean)
summary(modele_lag)

##
## Call:
## lm(formula = dep_delay ~ prev_dep_delay, data = data_lag_clean)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -146.99  -17.04  -13.30   -1.63  1289.59
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   11.405756   0.073686  154.79  <2e-16 ***
## prev_dep_delay 0.101143   0.001748   57.87  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 40.05 on 324482 degrees of freedom
## Multiple R-squared:  0.01021,    Adjusted R-squared:  0.01021
## F-statistic: 3348 on 1 and 324482 DF,  p-value: < 2.2e-16

```

Il existe une légère tendance à l'accumulation de retard sur un même avion : un vol en retard a un effet modeste sur le suivant.

Mais cette relation est faible, suggérant que d'autres facteurs (météo, trafic, organisation aéroportuaire, etc.) jouent un rôle beaucoup plus important.

4 Partie B : Programmation MapReduce

4.1 Écrire un programme MapReduce qui calcule le nombre de vols annulés par compagnie

```
# MAP : création d'une table avec 'carrier' et une colonne
# indiquant si le vol est annulé
map_17_df <- function(df) {
  df$annule <- is.na(df$arr_delay)
  return(df[, c("carrier", "annule")])
}

# REDUCE : somme des vols annulés par compagnie
reduce_17_df <- function(mapped) {
  aggregate(annule ~ carrier, data = mapped, sum, na.rm = TRUE)
}

# APPLICATION
mapped_17 <- map_17_df(jn)          # jn = ta table de données
result_17 <- reduce_17_df(mapped_17)
head(result_17)
```

	carrier <fctr>	annule <int>
1	9E	1166
2	AA	782
3	AS	5
4	B6	586
5	DL	452
6	EV	3065

6 rows

4.2 Retard maximal par compagnie

```
# MAP : remplace les NA de arr_delay par 0
map_18_df <- function(df) {
  df$arr_delay[is.na(df$arr_delay)] <- 0
  return(df[, c("carrier", "arr_delay")])
}

# REDUCE : calcul du maximum par compagnie
reduce_18_df <- function(mapped) {
  aggregate(arr_delay ~ carrier, data = mapped, max)
}

# APPLICATION
mapped_18 <- map_18_df(jn)
result_18 <- reduce_18_df(mapped_18)
head(result_18)

... 
```



	carrier <fctr>	arr_delay <dbl>
1	9E	744
2	AA	1007
3	AS	198
4	B6	497
5	DL	931
6	EV	577

6 rows

4.3 Compagnie avec le plus grand % de vols retardés mensuels

```
# MAP : créer une variable arr_del = TRUE si arr_delay >= 10
map_19_df <- function(df) {
  df$arr_del <- ifelse(!is.na(df$arr_delay) & df$arr_delay >= 10,
1, 0)
  return(df[, c("carrier", "month", "arr_del")])
}

# REDUCE : calcul du % de retards par (carrier, month), puis
sélection du max par mois
reduce_19_df <- function(mapped) {
  aggregated <- aggregate(arr_del ~ carrier + month, data = mapped,
FUN = mean)
  aggregated$pct_retards <- aggregated$arr_del * 100

  # Trouver pour chaque mois la ligne avec le % max
  result <- do.call(rbind, lapply(split(aggregated,
aggregated$month), function(df_month) {
    df_month[which.max(df_month$pct_retards), ]
  })))

  return(result[, c("carrier", "month", "pct_retards")])
}

# APPLICATION
mapped_19 <- map_19_df(jn)
result_19 <- reduce_19_df(mapped_19)
head(result_19)
```

	carrier <fctr>	month <int>	pct_retards <dbl>
1	OO	1	100.00000
2	EV	2	38.98615
3	YV	3	50.00000
4	F9	4	47.36842
5	F9	5	48.27586
6	FL	6	57.14286

6 rows